

Appunti completi

Python, Poetry, RAG, Agents e LangChain

Sintesi ragionata degli argomenti trattati nella conversazione

Nota iniziale

Ho confrontato gli argomenti indicati con tutta la conversazione. Gli argomenti richiesti sono presenti, ma durante il lavoro sono emersi anche altri temi importanti. Li ho aggiunti negli appunti per rendere il documento completo e utile per studiare.

Argomenti principali: Python - Poetry - RAG - Agents - LangChain

Argomenti aggiunti: FastAPI - Chainlit - ChromaDB - OpenAI API - Structured Outputs - Function Calling - Assistants API - LangGraph - Pydantic - Git/GitHub - .env - Laravel integration

Indice degli appunti

1. 1. Confronto tra argomenti richiesti e conversazione
2. 2. Mappa dei progetti costruiti
3. 3. Python: concetti usati nei progetti
4. 4. Poetry: gestione progetto e dipendenze
5. 5. FastAPI: API, router, request body e validazione
6. 6. RAG: recupero documenti, embeddings e ChromaDB
7. 7. Agents: tool, decisioni e orchestrazione
8. 8. LangChain e LangGraph
9. 9. OpenAI API: Structured Outputs, Function Calling, Assistants
10. 10. Chainlit e interfacce chat
11. 11. Progetto finale Travel Agent API
12. 12. Git, GitHub, .env e sicurezza
13. 13. Comandi utili e checklist finale
14. 14. Glossario rapido

1. Confronto tra argomenti richiesti e conversazione

Gli argomenti richiesti dall'utente erano Python, Poetry, RAG, Agents e LangChain. Tutti sono stati effettivamente trattati nella conversazione, ma non da soli: sono stati applicati in progetti pratici diversi. Per questo ho aggiunto anche gli argomenti collegati che servono a capire davvero il percorso fatto.

Argomento	Presente nella conversazione	Azioni negli appunti
Python	Si	Usato in tutti i progetti: file, funzioni, classi, moduli, typing, async, .env, test da terminale.
Poetry	Si	Usato per creare progetti, installare dipendenze, gestire ambienti virtuali e pyproject.toml.
RAG	Si	Usato soprattutto nel progetto HR Assistant con documenti, chunking, embeddings e ChromaDB.
Agents	Si	Trattati con Function Calling, Assistants API e Travel Agent con LangGraph ReAct Agent.
LangChain	Si	Usato nel progetto Travel Agent per tool, ChatOpenAI, prompt, parser e integrazione con LangGraph.
FastAPI	Aggiunto	Fondamentale nel selfwork Bookstore API e nel progetto finale Travel Agent API.
Chainlit	Aggiunto	Usato per HR Assistant e per interfacce chat Python.
ChromaDB	Aggiunto	Database vettoriale persistente usato nel progetto RAG HR Assistant.
OpenAI API	Aggiunto	Embeddings, Structured Outputs, Function Calling, Assistants API e modelli chat.
Pydantic	Aggiunto	Usato per validare request body, tool input e output strutturati.
Git/GitHub	Aggiunto	Usato per commit, push, .gitignore, README e consegna.

Conclusione del confronto

Gli argomenti richiesti corrispondono alla conversazione, ma per avere appunti completi ho aggiunto gli argomenti tecnici che hanno reso possibile i progetti: FastAPI, Chainlit, ChromaDB, OpenAI API, Pydantic, LangGraph, Git e sicurezza delle chiavi.

2. Mappa dei progetti costruiti

Durante la conversazione sono stati costruiti o studiati diversi progetti. Ognuno ha introdotto un pezzo diverso del mondo AI + Python.

Progetti e obiettivi

Voce	Spiegazione
HR Assistant	Sistema RAG per cercare candidati nei CV. Usa Chainlit, ChromaDB, embeddings OpenAI, chunking e document processing.
Structured Outputs	Esercizi per ottenere risposte JSON/Pydantic prevedibili dal modello, utili per ticket e ricerca immobiliare.
Function Calling Ristorante	Chatbot ristorante con tool Python per menu, disponibilit� e prenotazioni.
TechFix Assistant	Assistente OpenAI con file_search e funzione send_email

	simulata o reale tramite Resend.
Bookstore API	Selfwork FastAPI con path parameters, query parameters, request body e validation.
Travel Agent API	Progetto finale con FastAPI, LangChain, LangGraph, tools e compatibilit� con frontend Laravel.

Questa sequenza mostra una crescita naturale: prima Python e API, poi RAG, poi strumenti, poi agenti che decidono quali strumenti usare.

3. Python: concetti usati nei progetti

Python   il linguaggio di base che ha collegato tutti i progetti. Non   stato usato solo per scrivere script, ma per creare API, agenti, tool, classi di configurazione, validazioni e interfacce.

Concetti Python ricorrenti

Voce	Spiegazione
Moduli e package	I progetti usano cartelle come src/hr_assistant o src/travel_agent_api. Ogni cartella con __init__.py diventa un package importabile.
Import	Gli import collegano file diversi: routes importano services, services importano tools, tools importano librerie esterne.
Funzioni	Endpoint FastAPI, tool LangChain e funzioni di utilit� sono funzioni Python.
Classi	Config, Settings, Agent, modelli Pydantic e helper come ChatLogic sono classi.
Dizionari e liste	Le risposte JSON, i messaggi chat e i metadata dei documenti sono spesso dict e list.
Typing	Usato con int, str, float, bool, list, Optional e union come str None.
Async	Usato soprattutto in Chainlit e in alcune funzioni FastAPI, quando il codice deve lavorare con eventi o chiamate esterne.
Error handling	try/except nei tool per evitare crash quando SerpApi o servizi esterni falliscono.

```
# Esempio di package importabile
from travel_agent_api.services.agent_service import Agent

agent = Agent()
response = agent.run([
    {"role": "user", "content": "Cerco un hotel a Roma"}
])
print(response)
```

Idea chiave

Python viene usato come colla tra modelli AI, database, API web, file e interfacce. La parte importante non   solo la sintassi, ma l'organizzazione del codice in moduli piccoli e chiari.

4. Poetry: gestione progetto e dipendenze

Poetry   stato usato in tutti i progetti Python per creare l'ambiente, installare librerie e mantenere il progetto riproducibile.

Comandi Poetry fondamentali

Voce	Spiegazione
------	-------------

poetry new nome-progetto	Crea un nuovo progetto Python con struttura standard.
poetry add libreria	Installa una dipendenza e aggiorna pyproject.toml e poetry.lock.
poetry install	Installa tutte le dipendenze e anche il package locale. Utile quando compare ModuleNotFoundError sul package del progetto.
poetry env use python3.12	Forza Poetry a usare una versione specifica di Python.
poetry run comando	Esegue un comando dentro l'ambiente virtuale del progetto.
poetry check	Controlla che pyproject.toml sia valido.

```
poetry new travel-agent-api
cd travel-agent-api
poetry env use python3.12
poetry add fastapi uvicorn openai langchain-openai langchain-community langgraph pydantic
python-dotenv google-search-results
poetry install
poetry run python -c "import fastapi; print('ok')"
```

4.1 pyproject.toml e poetry.lock

pyproject.toml descrive il progetto: nome, versione, dipendenze e versione Python supportata. poetry.lock blocca le versioni esatte installate, in modo che il progetto sia riproducibile anche su un altro computer.

Regola pratica

Se usi Poetry, non serve requirements.txt. Su GitHub devi caricare pyproject.toml e poetry.lock. requirements.txt serve soprattutto nei progetti gestiti con pip.

5. FastAPI: API, router, request body e validazione

FastAPI è stato studiato prima con il selfwork Bookstore API e poi applicato nel progetto finale Travel Agent API. Serve per creare API web in Python, con documentazione automatica su /docs e /redoc.

Concetti FastAPI

Voce	Spiegazione
app = FastAPI()	Crea l'applicazione principale.
@app.get("/")	Crea una rotta GET.
Path parameters	Parti dinamiche dell'URL, per esempio /books/{book_id}.
Query parameters	Parametri dopo ?, per esempio /books/?limit=2.
Request body	Dati JSON inviati con POST o PUT.
Pydantic	Valida automaticamente i dati ricevuti.
APIRouter	Permette di separare le rotte in file diversi.
CORS	Permette al frontend Laravel su localhost:8000 di chiamare la API Python su 127.0.0.1:8080.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "API attiva"}

@app.get("/books/{book_id}")
def get_book(book_id: int):
    return {"book_id": book_id}

from pydantic import BaseModel, Field
```

```

class Book(BaseModel):
    title: str = Field(..., min_length=2, max_length=100)
    price: float = Field(..., gt=0)

@app.post("/books/")
def create_book(book: Book):
    return book.model_dump()

```

Swagger UI
FastAPI genera automaticamente la documentazione interattiva su /docs. Questo e stato utile per testare POST /chat/travel-agent nel progetto Travel Agent senza usare subito il frontend Laravel.

6. RAG: recupero documenti, embeddings e ChromaDB

RAG significa Retrieval Augmented Generation. L idea e semplice: prima cerco informazioni nei documenti, poi passo al modello solo le parti piu rilevanti, cosi la risposta e piu fondata sui dati disponibili.

Componenti di un sistema RAG

Voce	Spiegazione
Documenti sorgente	CV, PDF, Word, Excel, JSON, ZIP o altri file da indicizzare.
Document processor	Legge i file e li converte in testo pulito.
Chunking	Divide i documenti in pezzi piu piccoli.
Embeddings	Trasformano testo in vettori numerici confrontabili.
Vector database	Salva vettori e metadata. Nel progetto HR Assistant e stato usato ChromaDB.
Retrieval	Cerca i chunk piu simili alla domanda dell utente.
LLM	Genera la risposta usando domanda e contesto recuperato.

Flusso RAG semplificato:

- Documenti CV
- > estrazione testo
 - > chunking
 - > embeddings
 - > ChromaDB
- Domanda utente
- > embedding domanda
 - > ricerca chunk simili
 - > prompt con contesto
 - > risposta LLM

6.1 HR Assistant come esempio RAG

Nel progetto HR Assistant il sistema indicizza CV e documenti, poi risponde a domande sui candidati. Sono stati aggiunti persistenza ChromaDB, sincronizzazione incrementale, upload documenti da Chainlit, supporto multi-formato con Markdown e metadata come nome candidato, email, telefono, hash e tipo file.

Scelte importanti in HR Assistant

Voce	Spiegazione
ChromaDB PersistentClient	Permette di mantenere l indice anche dopo aver spento l app.
Hash file	Serve a capire se un documento e nuovo, modificato o invariato.
Chunking configurabile	Section, paragraph, fixed o semantic.
OpenAI text-embedding-3-small	Modello usato per creare embeddings.
Chainlit	Interfaccia chat con pulsanti, upload documenti e streaming risposte.

7. Agents: tool, decisioni e orchestrazione

Un agent è un sistema in cui il modello non genera solo testo, ma può decidere di usare strumenti esterni. Gli strumenti possono essere funzioni Python, API esterne, database, ricerche, file o altri servizi.

Differenza tra chatbot normale e agent

Voce	Spiegazione
Chatbot normale	Riceve testo e risponde con testo.
Agent	Riceve testo, ragiona sul compito, decide se usare tool, esegue azioni e poi risponde.
Tool	Funzione esterna che l'agent può chiamare, per esempio cerca hotel, cerca voli, invia email.
Orchestrazione	Coordinamento tra modello, memoria/conversazione, tool e risposta finale.

Flusso Agent semplificato:

```
Utente: "Cerco un hotel 4 stelle a Roma"
-> Agent riceve il messaggio
-> capisce che serve il tool hotels_finder
-> prepara parametri strutturati
-> chiama hotels_finder
-> riceve risultati
-> genera risposta finale leggibile
```

7.1 Esempi di agenti affrontati

Esempi pratici

Voce	Spiegazione
Ristorante Function Calling	Il modello usa funzioni per menu, disponibilità e prenotazioni.
TechFix Assistant	Assistant con file_search e funzione send_email.
Travel Agent API	Agent LangGraph con tool per storia, itinerario, voli e hotel.

8. LangChain e LangGraph

LangChain è un framework per costruire applicazioni basate su LLM. Nella conversazione è stato usato soprattutto nel progetto Travel Agent per creare tool, prompt, parser strutturati e integrazione con OpenAI. LangGraph è stato usato per creare un agente ReAct con create_react_agent.

Elementi LangChain usati

Voce	Spiegazione
@tool	Decora una funzione Python e la rende disponibile come strumento per il modello.
ChatOpenAI	Wrapper LangChain per usare modelli OpenAI.
ChatPromptTemplate	Costruisce prompt strutturati con messaggi system, human, user.
PydanticOutputParser	Converte una risposta del modello in un oggetto Pydantic strutturato.
create_react_agent	Crea un agent capace di scegliere e usare tool.

```
from langchain_core.tools import tool
```

```

@tool
def flights_finder(params):
    """Cerca voli usando SerpApi o dati simulati."""
    return {"message": "Ricerca voli simulata"}

from langgraph.prebuilt import create_react_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model_name="gpt-4o")
tools = [flights_finder, hotels_finder]
agent_executor = create_react_agent(model, tools)

```

Modalita test/prod

Nel progetto Travel Agent abbiamo aggiunto API_MODE. In test i tool restituiscono dati finti. In prod vengono usati OpenAI e SerpApi. Questa scelta permette di sviluppare senza costi e senza dipendere sempre da servizi esterni.

9. OpenAI API: Structured Outputs, Function Calling, Assistants

Oltre a LangChain, nella conversazione sono stati affrontati diversi modi di usare OpenAI in applicazioni Python.

Tecniche OpenAI affrontate

Voce	Spiegazione
Structured Outputs	Permettono di ottenere output prevedibili e validati con Pydantic, invece di testo libero.
Function Calling	Il modello sceglie una funzione da chiamare e restituisce argomenti strutturati.
Assistants API	Permette di creare assistant con thread, run, file_search e funzioni personalizzate.
Embeddings	Trasformano testo in vettori per RAG e ricerca semantica.
Streaming	Invio progressivo della risposta, usato in contesti chat come Chainlit.

9.1 Structured Outputs

Structured Outputs è utile quando Python deve usare direttamente la risposta del modello. Esempio: analisi ticket, estrazione parametri immobiliari o dati strutturati per un processo automatico.

```

class SupportTicketAnalysis(BaseModel):
    summary: str
    category: str
    priority: str
    needs_human_operator: bool

```

9.2 Function Calling

Function Calling consente al modello di non inventare un'azione, ma di chiedere a Python di eseguire una funzione reale. Nel progetto ristorante, il modello poteva chiamare menu, disponibilità o prenotazione.

Struttura tipica:

```

utente -> modello
modello -> tool_call con nome funzione e argomenti
Python -> esegue funzione
Python -> rimanda risultato al modello
modello -> risposta finale all'utente

```


9.3 Assistants API

Nel progetto TechFix Assistant sono stati usati Assistant, Thread, Run, file_search e funzione send_email. L'assistant poteva cercare nella knowledge base e richiedere l'invio email. Se RESEND_KEY era vuota, l'invio veniva simulato.

10. Chainlit e interfacce chat

Chainlit è stato usato per creare interfacce chat locali in Python. Nel progetto HR Assistant ha permesso di costruire una chat con pulsanti, upload documenti, reset indice, statistiche e streaming delle risposte.

Componenti Chainlit

Voce	Spiegazione
@cl.on_chat_start	Inizializza la sessione quando parte la chat.
@cl.on_message	Gestisce ogni messaggio utente.
Action	Pulsanti come Sincronizza CV, Statistiche indice, Reset indice.
AskFileMessage	Permette upload file da interfaccia.
user_session	Salva oggetti tra un messaggio e l'altro, per esempio database o logica chat.

11. Progetto finale Travel Agent API

Il progetto finale è la parte più importante della conversazione recente. È stato costruito da zero seguendo i PDF del corso, con FastAPI, LangChain, LangGraph, OpenAI e SerpApi.

Struttura finale Travel Agent API

Voce	Spiegazione
config.py	Centralizza API_MODE, OPENAI_API_KEY, SERPAPI_API_KEY, MODEL_NAME e origini CORS.
main.py	Crea FastAPI, configura CORS, include router e crea endpoint / e /health.
chat_router.py	Riceve messages e chiama Agent.run().
agent_service.py	Orchestra i tool e crea create_react_agent in modalità prod.
chain_historical_expert.py	Tool storico e culturale.
chain_travel_plan.py	Tool per itinerari.
flights_finder.py	Tool per voli con SerpApi o simulazione.
hotels_finder.py	Tool per hotel con SerpApi o simulazione.

Endpoint principale:

POST http://127.0.0.1:8080/chat/travel-agent

Body:

```
{
  "messages": [
    {"role": "user", "content": "Cerco un hotel 4 stelle a Roma centro"}
  ]
}
```

11.1 Personalizzazioni fatte

Refactoring e personalizzazioni

Voce	Spiegazione
config.py	Evita configurazioni sparse nei file.

API_MODE test/prod	Permette di testare senza chiamare OpenAI o SerpApi.
MODEL_NAME da .env	Permette di cambiare modello senza modificare codice.
Tracing	Print in router, Agent e tools per capire cosa viene eseguito.
Risposte simulate	Tool funzionanti anche senza chiavi API.
Validazione hotel_class	hotel_class deve essere tra 2 e 5.

11.2 Collegamento con Laravel

Il frontend Laravel usa Livewire e invia i messaggi a `http://127.0.0.1:8080/chat/travel-agent`. Per questo il backend Python deve girare sulla porta 8080 e permettere CORS da localhost:8000.

```
Python API:
poetry run uvicorn travel_agent_api.main:app --reload --port 8080
```

```
Laravel frontend:
composer install
npm install
npm run dev
php artisan serve
```

12. Git, GitHub, .env e sicurezza

Durante la conversazione è stato ripetuto più volte che le chiavi API non devono essere caricate su GitHub. Per questo si usa `.env` per i dati veri e `.env.example` come modello pubblico.

Regole sicurezza

Voce	Spiegazione
.env	Contiene chiavi vere. Non deve mai essere committato.
.env.example	Contiene nomi delle variabili, ma valori vuoti. Deve essere committato.
.gitignore	Deve contenere <code>.env</code> , <code>.venv</code> , <code>__pycache__</code> , <code>*.pyc</code> , <code>.DS_Store</code> , <code>.vscode</code> .
poetry.lock	Va committato, per rendere riproducibili le versioni.
requirements.txt	Non serve se il progetto usa Poetry, salvo richiesta specifica.

```
# .env.example
OPENAI_API_KEY=""
SERPAPI_API_KEY=""
API_MODE="test"
MODEL_NAME="gpt-4o"

# .gitignore
.env
.venv/
__pycache__/
*.pyc
.DS_Store
.vscode/
```

12.1 Commit semplici in italiano

Abbiamo scelto messaggi di commit semplici, comprensibili anche a chi non conosce l'inglese tecnico.

```
git commit -m "Creato progetto Travel Agent"
git commit -m "Aggiunta configurazione progetto"
git commit -m "Aggiunta API base"
git commit -m "Aggiunto tool storico"
git commit -m "Aggiunto tool piano viaggio"
git commit -m "Aggiunto tool ricerca voli"
git commit -m "Aggiunto tool ricerca hotel"
```

```
git commit -m "Aggiunto servizio Agent"
git commit -m "Collegato router ad Agent"
git commit -m "Aggiornato README finale"
```

13. Comandi utili e checklist finale

13.1 Check Python e Poetry

```
poetry env use python3.12
poetry install
poetry check
poetry run python --version
poetry run python -c "import fastapi, uvicorn, openai, langgraph, dotenv; print('Dipendenze ok')"
```

13.2 Check sintassi

```
poetry run python -m py_compile src/travel_agent_api/config.py
poetry run python -m py_compile src/travel_agent_api/main.py
poetry run python -m py_compile src/travel_agent_api/routes/chat_router.py
poetry run python -m py_compile src/travel_agent_api/services/agent_service.py
poetry run python -m py_compile src/travel_agent_api/tools/chain_historical_expert.py
poetry run python -m py_compile src/travel_agent_api/tools/chain_travel_plan.py
poetry run python -m py_compile src/travel_agent_api/tools/flights_finder.py
poetry run python -m py_compile src/travel_agent_api/tools/hotels_finder.py
```

13.3 Avvio API

```
poetry run uvicorn travel_agent_api.main:app --reload --port 8080
# poi apri:
# http://127.0.0.1:8080/docs
# http://127.0.0.1:8080/health
```

13.4 Check prima del push

```
git status
git ls-files | grep .env
# Deve comparire .env.example, non .env

git add .
git commit -m "Completato progetto Travel Agent"
git push
```

Checklist consegna Travel Agent

La consegna chiedeva parte Python, personalizzazione, tracing e README. Il progetto copre tutti questi punti: API FastAPI, LangChain/LangGraph, tool, modalita test/prod, print di tracing e README dettagliato.

14. Glossario rapido

Termini principali

Voce	Spiegazione
API	Interfaccia che permette a un programma di comunicare con un altro.
FastAPI	Framework Python per creare API rapide e documentate automaticamente.
Uvicorn	Server ASGI che esegue app FastAPI.
Poetry	Gestore di progetto, dipendenze e ambienti virtuali Python.

Pydantic	Libreria per validare dati usando classi Python e tipi.
RAG	Tecnica che combina recupero documenti e generazione con LLM.
Embedding	Rappresentazione numerica di un testo.
Vector database	Database che permette ricerca per similarita tra embeddings.
ChromaDB	Database vettoriale usato nel progetto HR Assistant.
Chunking	Divisione di un documento in parti piu piccole.
Agent	Sistema AI che puo usare tool per svolgere compiti.
Tool	Funzione o servizio che un agent puo chiamare.
LangChain	Framework per costruire applicazioni con LLM, prompt, tool e parser.
LangGraph	Framework per costruire agenti e flussi di lavoro con stato.
ReAct Agent	Agente che alterna ragionamento e azione tramite tool.
Structured Outputs	Output del modello in formato strutturato e validabile.
Function Calling	Meccanismo in cui il modello sceglie una funzione e argomenti da chiamare.
Assistants API	API OpenAI basata su assistant, thread, run, strumenti e file_search.
CORS	Regola browser che controlla quali frontend possono chiamare un backend.
.env	File locale con variabili sensibili, non da caricare su GitHub.
.env.example	Template pubblico senza chiavi vere.

Conclusione

Questi appunti riuniscono l'intero percorso pratico della conversazione: dai fondamenti di Python e Poetry fino a RAG, Function Calling, Assistants, FastAPI, LangChain e Agents. Il filo conduttore è stato costruire progetti reali, testabili e documentati.

Il concetto più importante da ricordare è questo: un'applicazione AI moderna non è solo una chiamata a un modello. È un sistema composto da API, configurazione, dati, tool, validazione, logging/tracing, sicurezza e documentazione.